Normalization and Functional Dependencies

Designing tables that avoid redundancy

1. Why Normalize?

Poorly designed tables repeat the same facts in many rows. Normalization is the step-by-step process of splitting a table into smaller, well-structured tables so that every fact is stored exactly once.

2. Update Anomalies

Consider one table *StudentCourse(Roll, Name, CourseID, CourseName, Instructor)*, where a student's name and a course's name are repeated in every row that mentions them. This causes three kinds of anomaly:

- **Insertion anomaly:** a new course cannot be added until at least one student enrolls in it.
- **Update anomaly:** renaming a course means changing many rows; missing one leaves inconsistent data.
- **Deletion anomaly:** deleting the last student of a course also erases all information about that course.

3. Functional Dependency (FD)

A functional dependency **X** $\rightarrow$ **Y** means that whenever two rows agree on attribute set X, they must also agree on attribute set Y. In other words, X determines Y. For example, *Roll* $\rightarrow$ *Name* and *CourseID* $\rightarrow$ *CourseName*.

- **Trivial FD:** Y is a subset of X (for instance, {Roll, Name} $\rightarrow$ Name).
- **Non-trivial FD:** Y contains at least one attribute that is not in X.
- **Full dependency:** Y depends on all of X, not on a proper subset of it.
- **Transitive dependency:** X $\rightarrow$ Y and Y $\rightarrow$ Z, so X $\rightarrow$ Z only through Y.

4. Armstrong's Axioms

- **Reflexivity:** if Y is a subset of X, then X $\rightarrow$ Y.
- **Augmentation:** if X $\rightarrow$ Y, then XZ $\rightarrow$ YZ for any Z.
- **Transitivity:** if X $\rightarrow$ Y and Y $\rightarrow$ Z, then X $\rightarrow$ Z.

5. Attribute Closure

The closure X^+ is the set of all attributes that X can determine using the given FDs. If X^+ contains every attribute of the relation, then X is a **super key**. Closure is the standard way to find candidate keys.

6. First Normal Form (1NF)

Every attribute holds only atomic (indivisible) values, and there are no repeating groups. A cell such as *Phone = '98xxx, 97xxx'* violates 1NF; the fix is to store one phone number per row, or move phones to their own table.

7. Second Normal Form (2NF)

The table is in 1NF and no non-key attribute depends on only part of a composite key. In *Enroll(Roll, CourseID, Name, Grade)* with key {Roll, CourseID}, the attribute *Name* depends on *Roll* alone, which is a partial dependency. Moving *Name* to a *Student(Roll, Name)* table fixes it.

8. Third Normal Form (3NF)

The table is in 2NF and has no transitive dependency of a non-key attribute on the key. If *Employee(EmpID, DeptID, DeptName)* has EmpID → DeptID and DeptID → DeptName, then DeptName depends on the key only through DeptID. Splitting into *Employee(EmpID, DeptID)* and *Dept(DeptID, DeptName)* gives 3NF.

9. Boyce-Codd Normal Form (BCNF)

A stricter form of 3NF: for every non-trivial FD X → Y, X must be a super key. BCNF removes the remaining anomalies that arise when a non-key attribute determines part of a candidate key.

10. Summary of the Normal Forms

Form	Rule	Removes
1NF	Atomic values, no repeating groups	Multi-valued cells
2NF	1NF + no partial dependency	Dependence on part of a key
3NF	2NF + no transitive dependency	Non-key determining non-key
BCNF	Every determinant is a super key	Remaining FD anomalies

Rule of thumb: every non-key attribute must depend on *the key, the whole key, and nothing but the key*.

11. Decomposition: What Makes a Split Good?

Normalizing means decomposing a relation R into smaller relations. A decomposition is only acceptable if it keeps two properties:

- **Lossless join:** joining the pieces back together returns exactly the original rows, with no extra (spurious) tuples. A binary split of R into R1 and R2 is lossless if $R1 \cap R2$ is a key of R1 or of R2.
- **Dependency preservation:** every original FD can be checked inside a single smaller table, without joining.

Decomposition into BCNF is always lossless but may lose dependency preservation. Decomposition into 3NF can always achieve both.

12. Worked Example

Take R(A, B, C, D) with FDs: $A \rightarrow B$, $B \rightarrow C$. The only candidate key is {A, D}, since $A^+ = \{A, B, C\}$ misses D.

- $A \rightarrow B$ is a partial dependency (A is part of the key), so split R into R1(A, B, C) and R2(A, D). This is 2NF.
- In R1, $B \rightarrow C$ is a transitive dependency, so split R1 into R11(B, C) and R12(A, B). This is 3NF and BCNF.
- Final tables: R11(B, C), R12(A, B), R2(A, D). Joining on B and then on A rebuilds the original data.

13. Beyond BCNF

- **Multivalued dependency ($X \twoheadrightarrow Y$):** X determines a whole set of Y values independently of the other attributes. Example: a person has several phone numbers and several email addresses, stored in one table.
- **4NF:** BCNF plus no non-trivial multivalued dependencies; fixes the phone and email style of redundancy.
- **5NF:** no join dependency that is not implied by the candidate keys; rarely needed in practice.

14. When to Denormalize

Highly normalized schemas need many joins. In read-heavy systems such as reporting databases, designers sometimes **denormalize** on purpose: they copy a column into another table, or pre-compute a total, to avoid an expensive join. The trade-off is faster reads against extra storage and the risk of inconsistent copies, so it is a deliberate decision made after the schema is already correct, never a substitute for normalizing in the first place.